```
import numpy as np
import pandas as pd
from sklearn.datasets import load_wine
import seaborn as sns
import matplotlib.pyplot as plt
```

```
wine = load_wine()
```

```
print(wine)
```

```
{'data': array([[1.423e+01, 1.710e+00, 2.430e+00, ..., 1.040e+00, 3.920e+00,
        1.065e+03],
       [1.320e+01, 1.780e+00, 2.140e+00, ..., 1.050e+00, 3.400e+00,
        1.050e+03],
       [1.316e+01, 2.360e+00, 2.670e+00, ..., 1.030e+00, 3.170e+00,
        1.185e+03],
       ...,
       [1.327e+01, 4.280e+00, 2.260e+00, ..., 5.900e-01, 1.560e+00,
        8.350e+02],
       [1.317e+01, 2.590e+00, 2.370e+00, ..., 6.000e-01, 1.620e+00,
        8.400e+02],
       [1.413e+01, 4.100e+00, 2.740e+00, ..., 6.100e-01, 1.600e+00,
        5.600e+02]]), 'target': array([0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
       0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
       0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 1, 1, 1, 1, 1, 1,
       1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
       1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
       1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 2, 2,
       2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2,
       2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2,
       2, 2]), 'frame': None, 'target_names': array(['class_0', 'class_1', 'class_2'], dtype='<U7'), 'DESCR': '.. _wine_dataset:
```

```
df = pd.DataFrame(data = wine.data,columns = wine.feature_names)
```

```
df['target'] = wine.target
```

```
df.head()
```

	alcohol	malic_acid	ash	alcalinity_of_ash	magnesium	total_phenols	flavanoids	nonflavanoid_phenols	proanthocyanins	col
0	14.23	1.71	2.43	15.6	127.0	2.80	3.06	0.28	2.29	
1	13.20	1.78	2.14	11.2	100.0	2.65	2.76	0.26	1.28	
2	13.16	2.36	2.67	18.6	101.0	2.80	3.24	0.30	2.81	
3	14.37	1.95	2.50	16.8	113.0	3.85	3.49	0.24	2.18	
4	13.24	2.59	2.87	21.0	118.0	2.80	2.69	0.39	1.82	

```
df['target'].value_counts()
```

target	count
1	71
0	59
2	48

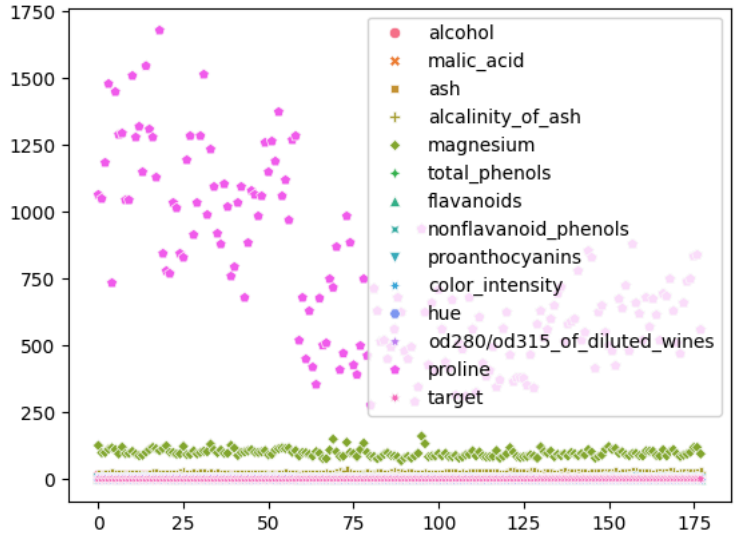
dtype: int64

```
df.shape
```

```
(178, 14)
```

```
sns.scatterplot(data= df)
```

```
<Axes: >
```




```
df.isnull().sum()
```

	0
alcohol	0
malic_acid	0
ash	0
alcalinity_of_ash	0
magnesium	0
total_phenols	0
flavanoids	0
nonflavanoid_phenols	0
proanthocyanins	0
color_intensity	0
hue	0
od280/od315_of_diluted_wines	0
proline	0
target	0

dtype: int64

```
x = df.drop('target',axis = 1)
y = df['target']
```

```
x
```

	alcohol	malic_acid	ash	alcalinity_of_ash	magnesium	total_phenols	flavanoids	nonflavanoid_phenols	proanthocyanins	c
0	14.23	1.71	2.43	15.6	127.0	2.80	3.06	0.28	2.29	
1	13.20	1.78	2.14	11.2	100.0	2.65	2.76	0.26	1.28	
2	13.16	2.36	2.67	18.6	101.0	2.80	3.24	0.30	2.81	
3	14.37	1.95	2.50	16.8	113.0	3.85	3.49	0.24	2.18	
4	13.24	2.59	2.87	21.0	118.0	2.80	2.69	0.39	1.82	
...	...	...	...	...	...	...	...	...	...	
173	13.71	5.65	2.45	20.5	95.0	1.68	0.61	0.52	1.06	
174	13.40	3.91	2.48	23.0	102.0	1.80	0.75	0.43	1.41	
175	13.27	4.28	2.26	20.0	120.0	1.59	0.69	0.43	1.35	
176	13.17	2.59	2.37	20.0	120.0	1.65	0.68	0.53	1.46	
177	14.13	4.10	2.74	24.5	96.0	2.05	0.76	0.56	1.35	

178 rows × 13 columns

```
y
```

	target
0	0
1	0
2	0
3	0
4	0
...	...
173	2
174	2
175	2
176	2
177	2

178 rows × 1 columns

dtype: int64

```
from sklearn.preprocessing import StandardScaler
```

```
scaler = StandardScaler()
```

```
scaler.fit(x)
```

```
▾ StandardScaler
StandardScaler()
```

```
standardized_data = scaler.transform(x)
```

```
x = standardized_data
y = df['target']
```

Start coding or generate with AI.

```
x
```

```
array([[ 1.51861254, -0.5622498 ,  0.23205254, ...,  0.36217728,
         1.84791957,  1.01300893],
       [ 0.24628963, -0.49941338, -0.82799632, ...,  0.40605066,
```

```
         1.1134493 ,  0.96524152],
       [ 0.19687903,  0.02123125,  1.10933436, ...,  0.31830389,
         0.78858745,  1.39514818],
       ...,
       [ 0.33275817,  1.74474449, -0.38935541, ..., -1.61212515,
        -1.48544548,  0.28057537],
       [ 0.20923168,  0.22769377,  0.01273209, ..., -1.56825176,
        -1.40069891,  0.29649784],
       [ 1.39508604,  1.58316512,  1.36520822, ..., -1.52437837,
        -1.42894777, -0.59516041]])
```

```
y
```

	target
0	0
1	0
2	0
3	0
4	0
...	...
173	2
174	2
175	2
176	2
177	2

178 rows × 1 columns

dtype: int64

```
from sklearn.model_selection import train_test_split
```

```
x_train,x_test,y_train,y_test = train_test_split(x,y,test_size = 0.2,random_state=2)
```

```
x_train
```

```
array([[-0.51957465, -0.93926832, -0.97420996, ..., -1.65599853,
        -1.49956991, -0.34040091],
       [-1.91542406,  0.05713777,  0.19549913, ..., -0.69078402,
         1.09932487, -0.38816832],
       [-1.13720713, -0.90336179, -0.24314178, ...,  0.49379744,
         0.84508517, -0.38816832],
       ...,
       [ 0.60451647, -0.60713296, -0.46246223, ...,  0.09893695,
         0.23773476, -0.87539586],
       [ 0.77745356, -0.47248348,  1.21899459, ...,  1.41513857,
         0.37897904,  1.79320989],
       [ 0.71569031,  0.21871714,  1.18244118, ..., -0.95402434,
        -1.1464592 ,  0.0098934 ]])
```

```
y_train
```

	target
154	2
120	1
89	1
92	1
156	2
...	...
43	0
22	0
72	1
15	0
168	2

142 rows × 1 columns

dtype: int64

```
from sklearn.svm import SVC,SVR
```

```
model = SVC(kernel = 'linear',C = 1000)
```

```
help(SVC)
```

```
Help on class SVC in module sklearn.svm._classes:

class SVC(sklearn.svm._base.BaseSVC)
 |  SVC(*, C=1.0, kernel='rbf', degree=3, gamma='scale', coef0=0.0, shrinking=True, probability=False, tol=0.001, cache_size=26
 |
 |  C-Support Vector Classification.
 |
 |  The implementation is based on libsvm. The fit time scales at least
 |  quadratically with the number of samples and may be impractical
 |  beyond tens of thousands of samples. For large datasets
 |  consider using :class:`~sklearn.svm.LinearSVC` or
 |  :class:`~sklearn.linear_model.SGDClassifier` instead, possibly after a
 |  :class:`~sklearn.kernel_approximation.Nystroem` transformer or
 |  other :ref:`kernel_approximation`.
 |
 |  The multiclass support is handled according to a one-vs-one scheme.
 |
 |  For details on the precise mathematical formulation of the provided
 |  kernel functions and how `gamma`, `coef0` and `degree` affect each
 |  other, see the corresponding section in the narrative documentation:
 |  :ref:`svm_kernels`.
 |
 |  Read more in the :ref:`User Guide <svm_classification>`.
 |
 |  Parameters
 |  ----------
 |  C : float, default=1.0
 |      Regularization parameter. The strength of the regularization is
 |      inversely proportional to C. Must be strictly positive. The penalty
 |      is a squared l2 penalty.
 |
 |  kernel : {'linear', 'poly', 'rbf', 'sigmoid', 'precomputed'} or callable,          default='rbf'
 |      Specifies the kernel type to be used in the algorithm. If
 |      none is given, 'rbf' will be used. If a callable is given it is used to
 |      pre-compute the kernel matrix from data matrices; that matrix should be
 |      an array of shape ``(n_samples, n_samples)``. For an intuitive
 |      visualization of different kernel types see
 |      :ref:`sphx_glr_auto_examples_svm_plot_svm_kernels.py`.
 |
 |  degree : int, default=3
 |      Degree of the polynomial kernel function ('poly').
 |      Must be non-negative. Ignored by all other kernels.
 |
 |  gamma : {'scale', 'auto'} or float, default='scale'
 |      Kernel coefficient for 'rbf', 'poly' and 'sigmoid'.
 |
 |      - if ``gamma='scale'`` (default) is passed then it uses
 |        1 / (n_features * X.var()) as value of gamma,
 |      - if 'auto', uses 1 / n_features
```

```
 |      - if float, must be non-negative.
 |
 |      .. versionchanged:: 0.22
 |         The default value of ``gamma`` changed from 'auto' to 'scale'.
 |
 |  coef0 : float, default=0.0
 |      Independent term in kernel function.
 |      It is only significant in 'poly' and 'sigmoid'.
 |
```

```
train = model.fit(x_train,y_train)
```

```
from sklearn.metrics import mean_squared_error,mean_absolute_error,r2_score,accuracy_score
```

```
x_train_pred = model.predict(x_train)
```

```
x_train_accuracy = accuracy_score(x_train_pred,y_train)
```

```
x_train_accuracy
```

```
1.0
```

```
x_test_pred = model.predict(x_test)
x_test_accuracy = accuracy_score(x_test_pred,y_test)
```

```
x_test_accuracy
```

```
0.9722222222222222
```

```
input_data =[13.27,4.28,2.26,20,120,1.59,0.69,0.43,1.35,10.2,0.59,1.56,835]

#changing the input_data to numpy array
input_data_as_numpy_array = np.asarray(input_data)

#reshape the arrat as are predicting for one instance
input_data_reshape = input_data_as_numpy_array.reshape(1,-1)

#standardized the input_data
std_data= scaler.transform(input_data_reshape)

print(std_data)

prediction = model.predict(std_data)
print(prediction)
if (prediction[0]==1):
    print('The person is non-diabetic')
elif(prediction[0] == 2):
```